

动态树分治

成都七中 nzhtl1477

常用的树分治方法

- 1.点分治
- 2.链分治
- 3.边分治
- 4.link cut tree
- 5.top tree

Notice

- 目前我见过的类树分治的东西有：
- 点分治
- 边分治
- 链分治 --- 树链剖分 --- 树上启发式合并
- lct维护子树
- 静态lct维护子树
- 类top tree
- Top tree
- 静态top tree

Note

- 其实会点分治/链分治就够用了
- Top tree类的东西现在基本上没人会的，所以不用管
- 事实是top tree是目前我觉得的最通用化的树分治处理数据结构

静态树分治

- 每次找一个分治中心，计算跨过分治中心的所有答案，然后删去分治中心，继续递归分治下去计算
- 把所有分治中心的答案合并起来就是全局的答案

点分治

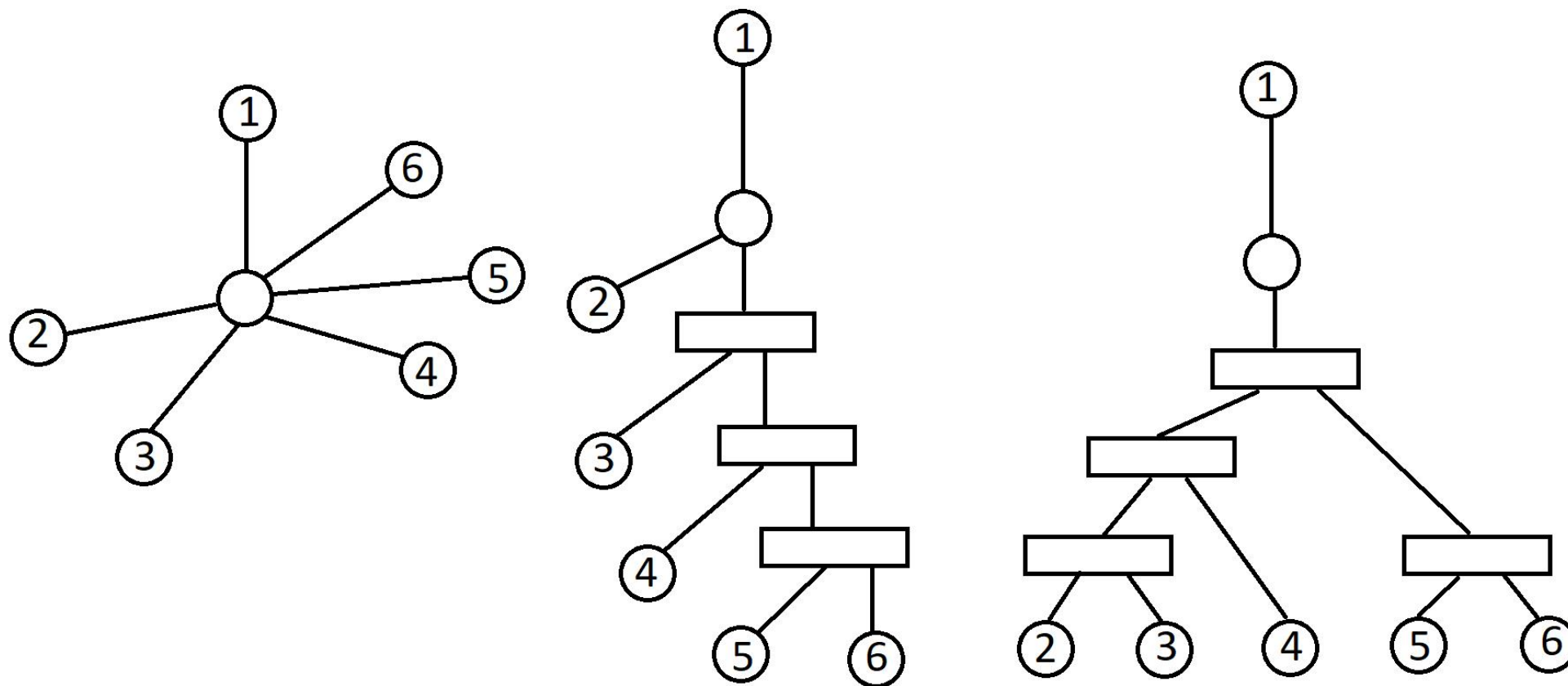
- 可以看作是每次删去一个点之后继续分治下去
- 每次找树的一个重心，然后递归每个子树
- 这些找到的重心构成一个树形结构
- 因为这个分治中心构成的树的深度 $O(\log n)$
- 所以还是在每个分治中心维护一个数据结构
- 修改和查询都暴力跳就可以了

边分治

- 可以看作是每次删去一条边之后继续分治下去
- 删去一条边之后树会分成两个子树，删去的边需要保证让两边的子树size尽可能相近
- 直接朴素地在这个树上做可能复杂度不正确，需要进行三度化

三度化

- 对于每个度数 >3 的节点，可以通过加虚点的方法让该节点度数变为3



三度化

- 注意三度化只是可以解决部分树点度数过大的问题，因为其本质并不是对点度数进行了分治，而仅仅是改变了树的结构

链分治

- 基于HLD的分治结构
- 可以看作是每次删去一条重链之后继续分治下去
- 每个重链头开一个数据结构
- 每次修改的时候跳重链，在每个重链头的数据结构上进行修改
- 查询同理

动态树分治

- 静态树分治每次选取的分治中心构成了一个树形结构
- 我们把这个树建出来，然后每次修改的时候可以在上面维护信息，这样就不用每次重新做一遍了
- 类比：
- 比如全局最大子段和可以分治做，单点修改全局最大子段和可以线段树来做，线段树就是把序列上的分治结构存下来了，每次在上面维护信息，从而避免了大量的重复计算

动态树分治

- 分治信息：只有 $O(1)$ 的信息量，可以高效合并，有交换律，也就是一个半群
- 树链剖分：支持链/子树修改，查询链/子树分治信息
- 先看几个简单的树上问题？

Luogu4216 [SCOI2015]情报传递

- 给你一棵树
- 1 x: 让一个点从当前时刻开始，每秒操作点权++
- 2 x y c: 查询一条链中有多少点的点权大于c
- 其中每秒操作点权++就是指我每操作一次，无论是否和那个点有关，那个点权值都会++
- 1操作对于每个点只会开始一次

Solution

- 考虑我们肯定不能每次把1操作的点++, 这样复杂度不对
- 看看2操作吧

Solution

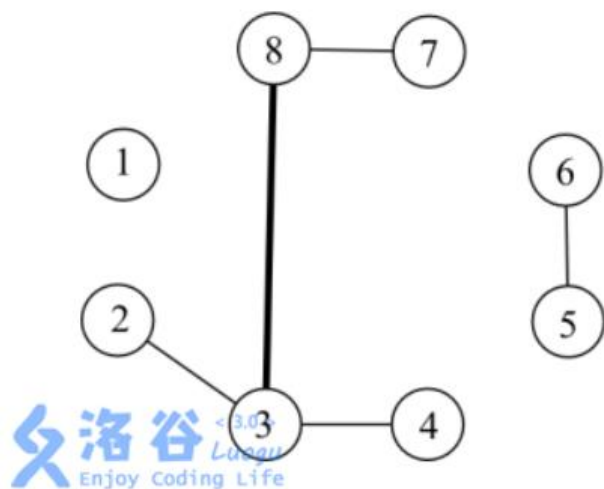
- 查询 $x \rightarrow y$ 中有多少点权 $\geq c$ 的点
- 而点权是一秒+1
- 所以等价于查 $x \rightarrow y$ 中有多少当前时间-开始时间 $\geq c$ 的点
- 即查询有多少点满足当前时间- $c \geq$ 开始时间
- 相当于查一条链中小于一个数个数的数
- 这样可以树链剖分+树状数组做到 $O(m \log^2 n)$

Solution2

- 进一步研究
- 如果第 i 次操作是 $1\ x$, 则相当于第 i 时刻 x 点开始
- 可以把操作 1 当成第 i 时刻的时候点 x 的权值从 0 变成了 1
- 如果第 i 次操作是 $2\ x\ y\ c$, 则相当于查询链 $x \rightarrow y$ 中开始时间 $\leq i-c$ 的点的个数
- 等价于查询第 $i-c$ 时刻链 $x \rightarrow y$ 上的和
- 通过之前讲的树上差分+树状树组直接维护即可
- $O(m \log n)$

Luogu4219 [BJOI2014]大融合

小强要在 N 个孤立的星球上建立起一套通信系统。这套通信系统就是连接 N 个点的一个树。这个树的边是一条一条添加上去的。在某个时刻，一条边的负载就是它所在的当前能够联通的树上路过它的简单路径的数量。

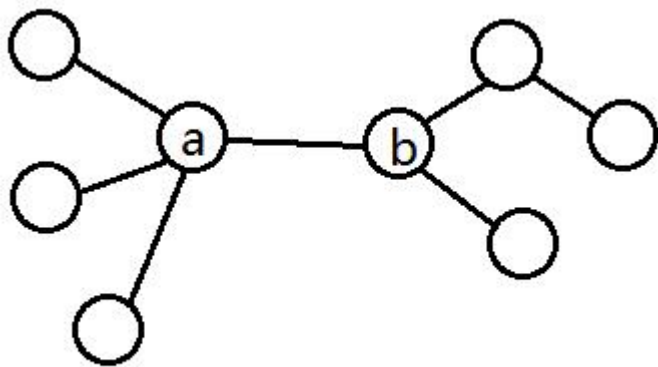


例如，在上图中，现在一共有5条边。其中， $(3, 8)$ 这条边的负载是6，因为有以下六条简单路径： $2 - 3 - 8$ ， $2 - 3 - 8 - 7$ ， $3 - 8$ ， $3 - 8 - 7$ ， $4 - 3 - 8$ ， $4 - 3 - 8 - 7$ 路过了 $(3, 8)$ 。

现在，你的任务就是随着边的添加，动态的回答小强对于某些边的负载的询问。

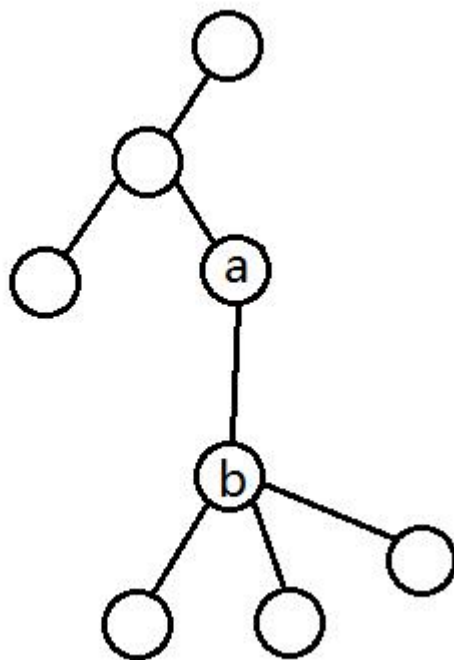
Solution

- 可以发现每次查询点a, b两端构成的简单路径个数
- 即等价于a不经过b的子树大小和b不经过a的子树大小的乘积
- 因为任意左边一个点和右边一个点都有唯一的而且不同的一条路径



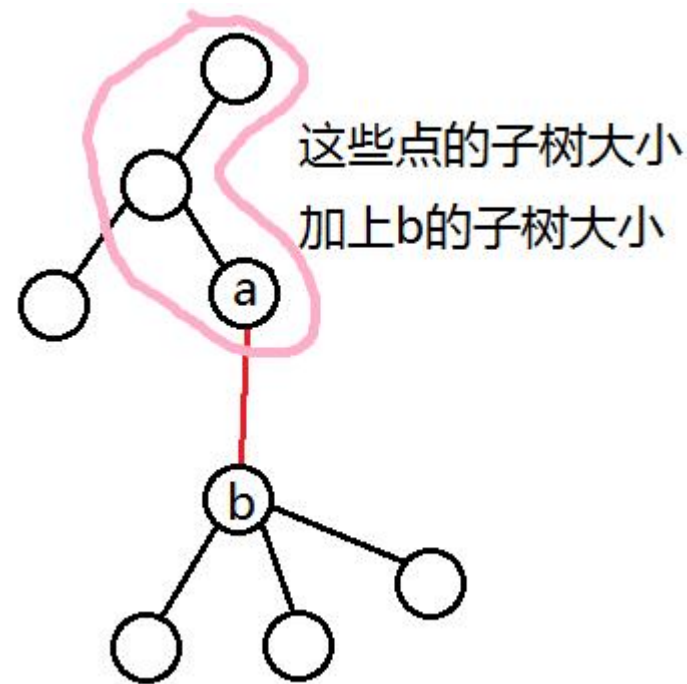
Solution

- 于是考虑离线，先把这棵树建出来，每个点维护子树大小
- 假设a是b的父亲，则a和b之间路径的答案为(a所在联通块大小 - b子树大小) * b子树大小



Solution

- 每个点维护子树大小，用并查集维护每个联通块的大小
- 则每次连接一条边的时候，等价于把一条链上的子树大小都加上一个值
- 比如连接a和b就是把：
 - a所在联通块里面深度最低的那个点到a
 - 这一条链加一个数
 - 深度最低的那个点用并查集维护即可



Solution

- 问题转换为链加，单点求值
- 用树状数组轻松维护即可
- $O(q \log n)$

Luogu4211 [LNOI2014]LCA

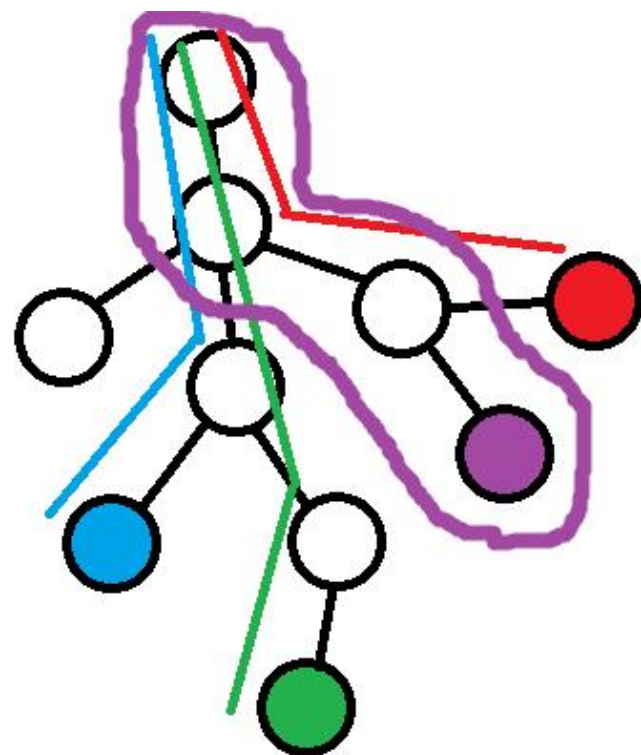
- 给出一个 n 个节点的有根树（编号为 0 到 $n-1$ ，根节点为 0 ）。一个点的深度定义为这个节点到根的距离 $+1$ 。
设 $dep[i]$ 表示点 i 的深度， $LCA(i,j)$ 表示 i 与 j 的最近公共祖先。
有 q 次询问，每次询问给出 $l\ r\ z$ ，求 $\sum_{l \leq i \leq r} dep[LCA(i,z)]$ 。
（即，求在 $[l,r]$ 区间内的每个节点 i 与 z 的最近公共祖先的深度之和）

Solution

- 首先将查询差分
- $(l,r) \rightarrow (1,r) - (1,l-1)$
- 然后考虑给一个点，怎么求其到一个前缀的点的LCA的深度和

Solution

- 可以把每个点按顺序依次插入，每次插入把这个点到根的路径++，查询z到这些点各自的lca的深度和即查询z到根路径的和
- 如图，绿色，红色，蓝色的点被插入，
- 查询紫色的点
- $O(m \log^2 n)$



Codechef DGCD

- 给出一棵点权树，有M次操作：
 - 1、询问一条路径上的GCD
 - 2、将一段路径上的点权加上d

Solution

- $\gcd(a, b) = \gcd(a - b, b)$
- 所以我们可以维护差分后的序列，区间加变成了单点修改，就可以维护了

Luogu2056 [ZJOI2007]捉迷藏

- 树，点权0, 1
- 每次修改一个点点权，询问两个最远1的距离

Solution

- 对于传统树分治做法，就是在每个树分治的分治中心上开个堆维护一下：
- 每个子树一个堆维护到其的最大距离
- 再开个堆维护上面那个堆的前2大值
- 再开个堆维护全局的最大值
- 然后每次暴力更新即可，总复杂度 $O((n + m)\log^2 n)$

Solution

- 还有一种括号序列的做法
- 本质上是ETT在这个题上的特殊化，这里就不介绍了

Bzoj2566 xmastree

- 有一棵含 N 个结点的树，树上每条边 (a_i, b_i) 都有一个权值 w_i 。树上每个结点涂有一个初始颜色 c_i 。现在有很多次修改操作，第 i 次修改会将结点 x_i 的颜色修改成 y_i 。请在所有修改前和每次修改之后输出一个数，表示对应时刻最近的同色结点对的距离。
- 其中，距离定义为树上两点的最短路的距离。最短路按边的权值 w_i 计算。

Solution

- 和之前那个题差不多，对每个颜色分别开堆维护即可，大概就是
`map < int , priority_queue < int > >` 这样的写法

Luogu6329 震波

- 树，点权
- 1.修改一个点的点权
- 2.查询距离一个点 $\leq k$ 的所有点的点权和

Solution

- 裸题
- 开一个数据结构，维护离分治中心距离为1...的所有点的点权和
- 为了防止算重，可以再开一个数据结构，维护每个点到分治中心的父亲的负贡献即可差分掉

Bzoj4372 烁烁的游戏

- 树, 点权
- 1.修改距离一个点 $\leq k$ 的所有点的点权和
- 2.查询一个点的点权

Solution

- 和前一题有啥区别

Luogu5311 [Ynoi2011]成都七中

- 音无彩名给了你一棵 n 个节点的树，每个节点有一种颜色，有 m 次查询操作
- 查询操作给定参数 $l\ r\ x$ ，需输出：
- 将树中编号在 $[l,r]$ 内的所有节点保留， x 所在联通块中颜色种类数
- 每次查询操作独立

Solution

- 每次查询的时候
- 考虑在点分树上找到深度最浅的一个点，满足这个点被保留区间点 x 所在连通块包含
- 这个节点一定能被找到
- 然后我们可以把根固定为这个点来进行统计

Solution

- 转换为
- 给定一棵有根树
- 每次查询保留区间点，根所在连通块的颜色数

Solution

- 对每个颜色分别考虑贡献
- 一个点在保留区间颜色的时候和根连通等价于这个点到根路径上 $[\min, \max]$ 的区间是区间 $[l, r]$ 的子区间
- 问题就转化为了，平面上有若干个点，询问某个点左下角的点的颜色数
- 这个可以离线 $O(\log n)$ 解决，前面点分治是 $O(\log n)$ 代价
- 总复杂度 $O(n \log^2 n)$

floj307 不可知圆环

Description

纱绫发现，『不可知圆环』不仅可以进行传送，如果在传送中关闭，还可以将物品切断。

纱绫决定试验一下这个能力。她找到了一个 n 个节点的树，对于每条边都决定切断或不切断，这样一共有 2^{n-1} 种方案。纱绫想知道，有多少种方案使得点 1 所在连通块的大小为 k 。答案对 1811939329 取模。

Solution

- 如何合并两个子图的答案？
- $f[i]$ 表示合并上来的连通块大小为 i 的方案数
- 用 $f1$ 和 $f2$ 合并出 g :
- $g[i] = \sum f1[i] + f2[i-j]$
- 这个实际上就是
- $\text{for}(i, j) \quad g[i+j] += f1[i] + f2[j]$
- 是一个卷积形式，考虑树分治+FFT维护

Solution

- 处理树的问题通常可以考虑分治，而点分治在这道题中会有一些不便，考虑进行链分治。
- 先对树进行重链剖分，每次处理一条重链。
- 先递归下去求出每个轻儿子的答案数组。对于一个重链上的点，合并其轻儿子可以用分治 FFT 做到 $O(n \log^2 n)$

Solution

- 这样每个重链上的点都有一个答案数组。
- 考虑合并重链上的点。如果按深度从小到大对重链编号，则连通块在重链上的部分一定是一段前缀，可以对重链进行分治 FFT。一个前缀要么是左半的一部分，要么是左半加右半的一部分，所以分治时可以维护两个数组，一个是包含左端点的答案，一个是同时包含左右端点的答案。
- 时间复杂度为 $O(n \log^3 n)$ ，但常数很小。

Solution

- 树分治+重链分治我们会想到什么?
- 树链剖分
- 树链剖分是 $2\log$ 的, 因为划分不均匀
- 可以使用更优的结构做到 $1\log$
- 于是这题可以做到 $2\log$

静态lct

- 很多问题并不涉及到树形态的修改，所以用lct维护会常数比较大
- 可以考虑将lct给“静态化”
- 发现HLD的问题是虽然跳轻边的次数是 $O(\log n)$ 的，但是在重链上表现不佳
- 因为HLD是对于每条重链局部平衡的，并不能和全局的结构很好地配合起来

静态lct

- 对于重链我们建一个平衡的线段树，而是建一个基于biased dictionary problem的bst
- 将每个节点赋予额外权值为其轻儿子子树size和+1
- 对于每条重链我们找该重链的带权重心，以该节点作为bst的根，两边做为左右子树建bst
- 轻边不改变
- 可以证明这样的结构我们每次跳a层，子树size一定会增大 $1/b$ ，其中a和b是两个常数，好像是bst上跳三层一定增大一倍，所以深度是 $O(\log n)$ 的
- 这个挺好写的，缺点是静态结构无法动态改变树的形态

Luogu5314 [Ynoi2011]ODT

- 给一棵树，支持：
- 1.把一条链上所有点加上 k
- 2.查询距离一个点 ≤ 1 的所有点的点权 k th

Luogu5314 [Ynoi2011]ODT

- 对这题的评价: (5-8)/11

Solution1

- 暴力
- 总复杂度 $O(nm)$
- 期望得分：20

Solution2

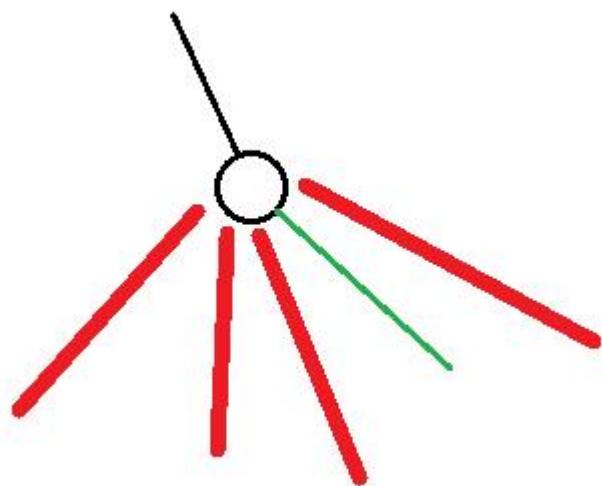
- 对点的度数进行根号分治
- 度数 $>\sqrt{n}$ 的点有 $\sqrt{n}$ 个, 其他点度数都 $<\sqrt{n}$
- 维护所有大点的数据结构
- 每次修改 $O(\sqrt{n} * \text{数据结构复杂度})$

Solution2

- 查询的时候如果是大点就直接查，如果是小点就是暴力查询一圈
- 这个是 $O(\sqrt{n} * \text{数据结构复杂度})$
- 对两个数据结构都用一个 $O(\sqrt{n}) - O(1)$ 平衡的分块
- 于是做到了 $O(m\sqrt{n})$ 的复杂度
- 期望得分：20-50

Solution3

- 考虑链分治
- 一个点距离 ≤ 1 的点里面，只有3个可能不是重链头
- 自己，父亲，轻儿子



Solution3

- 然后每次查询的时候暴力插入这3个 ($O(1)$ 个) 点
- 每次修改的时候把一条链上所有重链头的父亲都修改这个重链头的值
- 于是做到了查询 $O(\log n)$
- 修改 $O(\log^2 n)$
- 的复杂度, 期望得分: 80
- 看看能不能继续优化?

Solution4

- 考虑查询一个点的时候再处理经过这个点的所有修改
- 直觉上这样可以减少很多修改

Solution4

- 这样的复杂度似乎是均摊 $O((n + m)\log n)$?
- 证明1:
- 这个问题可以转换为:
- 有一颗边全是黑色的树
- 每次可以把一条链染色为黑色
- 或者把一个点一圈全染色为白色, 复杂度为一圈黑色的点数
- 证明: 复杂度为 $O(n + m)$

Fact

- 我问了一群大爷，大家都从直觉上觉得这个东西肯定是 $O(n + m)$ 的，但是都无法给出严谨的证明。。。

God



Oscar

搞一棵 x 层 x 叉树



Oscar

每当有一个叶子到根的路径都是白的



Oscar

就把她搞黑



Oscar

每当有一个节点所有指向儿子的边都是黑的



Oscar

就把她搞白



Oscar

然后可以 $n \cdot x$ 次内每次都改变至少 x 条边的状态



Oscar

并且最终状态和初始状态一样



Oscar

😞😞😞



Oscar

这样对不对啊

Oh no

- 我们的直觉其实很有可能是错的

WTF

- 那这样的话按照复杂度证明1, 这个的复杂度是什么呢?
- 相当于求 x^x 的反函数
- 这个的复杂度是 $e^{W(\ln(n))}$, W 是ProductLog函数
- 然后可以证明出
- $e^{W(\ln(n))} = \Theta(\log n / \log \log n)$

$$\left(\frac{\ln x}{\ln(\ln x)} \right)^{\frac{\ln x}{\ln(\ln x)}}$$

$$= e^{\frac{\ln x}{\ln(\ln x)} \ln \left(\frac{\ln x}{\ln(\ln x)} \right)}$$

$$= e^{\frac{\ln(x) (\ln(\ln x) - \ln(\ln(\ln x)))}{\ln(\ln x)}}$$

$$= e^{\ln(x) \left(1 - \frac{\ln(\ln(\ln x))}{\ln(\ln x)} \right)}$$

$$= x^{1 - \frac{\ln(\ln(\ln x))}{\ln(\ln x)}}$$

Solution4

- 如果按照这个问题来算
- 这个做法的复杂度是 $\Omega((n + m)\log^2 n / \log \log n)$
- 期望得分：100

Solution5

- 这个问题比原问题强，进一步弱化
- 有一颗 n 个点的树，边有颜色，初始全黑，支持把一条链上的边染黑，或者把一个点相邻的边染白(代价为 相邻黑边数 $\times \log(\text{度数})$)
求证 n 次操作代价为 $O(n \log n)$

Solution5

- 可以看到那个hack数据对这个做法的复杂度是：
- $O(\log n / \log \log n * \log(\log n / \log \log n)) < O(\log n / \log \log n * \log \log n) = O(\log n)$
- 失效了

Preknowledge

- 多序列kth的做法：
设有m个序列，长度为 $a[1], a[2], \dots, a[m]$ ，由m棵平衡树分别维护，
希望查出第k小的元素
要求序列中不含相同元素
不妨设 $k \leq a[1] + a[2] + \dots + a[m]$

Preknowledge

- 平衡树支持 $O(1)$ 找出rank在 $[71\%, 80\%]$ 之间的某个元素（不关心具体rank，只要在范围内即可），并按这个元素 $O(1)$ 分裂成比它小/大的部分（不包含这个元素）

Preknowledge

- 将每棵平衡树按对应序列长度大小加权，以某个rank在 $[71\%, 80\%]$ 的元素为key， $O(m)$ 求加权nth_element，将m棵树分成3组，第二组只有一棵树，第三组的key比第二组大，第一组的key比第二组小，第1,2组的序列长度之和至少占总序列长度之和的71%，第2,3组的序列长度之和至少占总序列长度之和的29%。于是第1,2组中小于等于本组key的元素个数占m个序列长度之和的50%以上，且这些元素都比第二组的key小；第2,3组中大于等于本组key的元素个数占m个序列长度之和的5.8%以上。将第2,3组中平衡树替换为小于本组key的部分，得到子问题，而总的序列长度减少了至少5.8%。
因此复杂度是 $O(m \log(a[1] + a[2] + \dots + a[m]))$

Solution5

- 对每个点，将轻边按子树size降序排序，分组，每组依次包含2,4,16,256,65536, ...条轻边，每组用平衡树维护
查kth的时候使用多序列kth的trick，复杂度是 $\log n$ 的
链加的时候特判重链，可以发现轻边上改平衡树的复杂度是 $\log n$ 的

Solution5

- 第 k 组的轻边至多有 $2^{(2^{(k-1)})}$ 条
轻边为第一组的情况修改是 $O(1)$ 的，这种情况每次链加只有 $O(\log n)$ 次
轻边不在第一组时，自顶向下经过第 k 组轻边，则子树大小减小至不超过原来的 $1/(2^{(2^{(k-2)})})$ ，修改复杂度是 $O(2^{(k-1)})$ ，以 $\log_2(\text{子树大小})$ 为势能可以证明单次链加的复杂度为 $O(\log n)$
重链部分把链加点查转化为点加子树求和，也是 $O(\log n)$ 的
- 时间复杂度 $O((n + m)\log n)$
- 空间复杂度 $O(n + m)$
- 期望得分：80-100

Luogu5529 [Ynoi2012]梦断SCOI2017

- 给一棵树，点权，支持：
- 1.单点修改
- 2.修改一个点所在同色块的颜色
- 3.查询一个点所在同色块的最浅点和最深点
- 颜色数 $c=30$

Luogu5529 [Ynoi2012]梦断SCOI2017

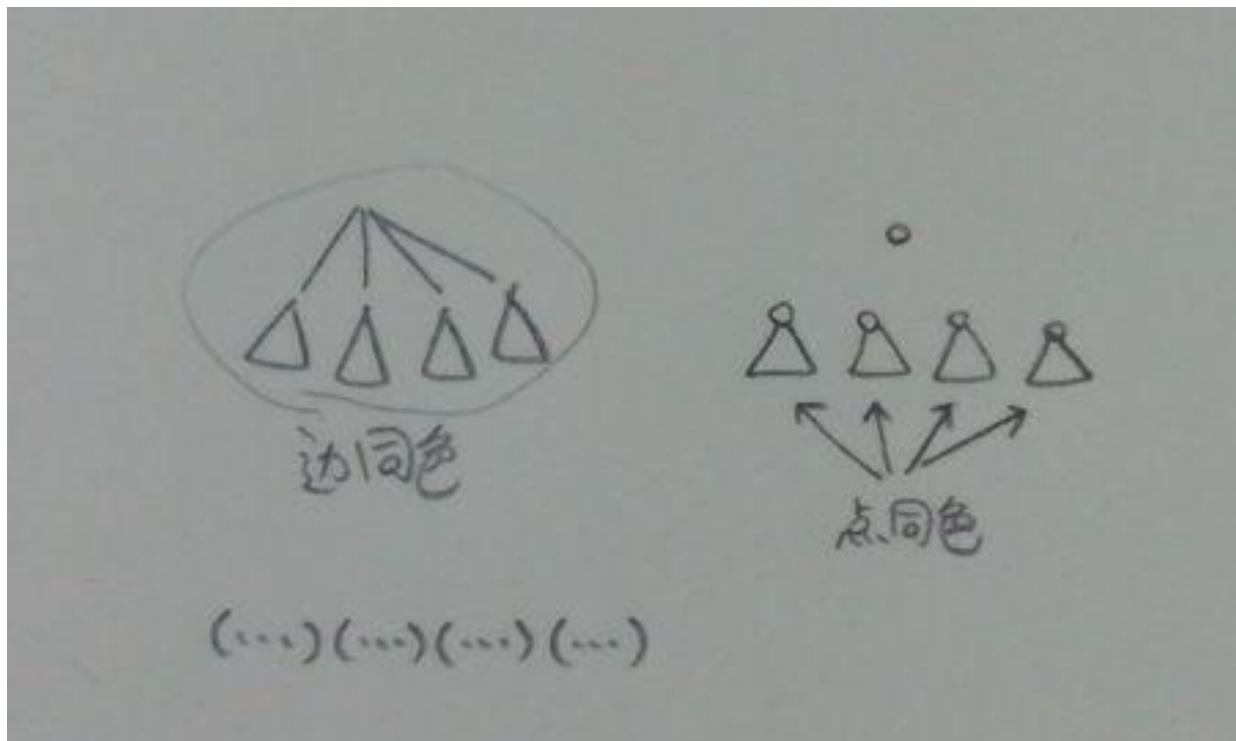
- 对这题的评价：6/11

Solution1

- 暴力
- 总复杂度 $O(nm)$
- 期望得分：20

Solution2

- 点同色连通块的括号序列对应于所在的边同色连通块的括号序列的一个区间，因为边同色连通块只比点同色连通块多一条边

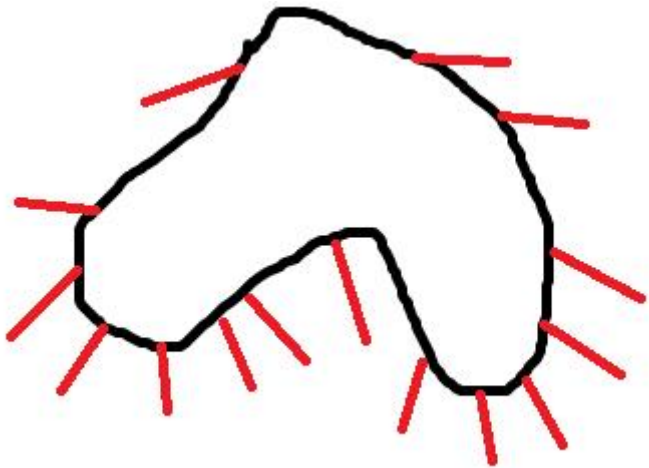


Solution2

- 因为点的联通块没有均摊，但是边的联通块有均摊，所以维护边的联通块而不是点的联通块
- 可以发现改变一条边的颜色只影响 $O(1)$ 个边同色连通块，而点同色连通块可能影响 $O(n)$ 个

Solution2

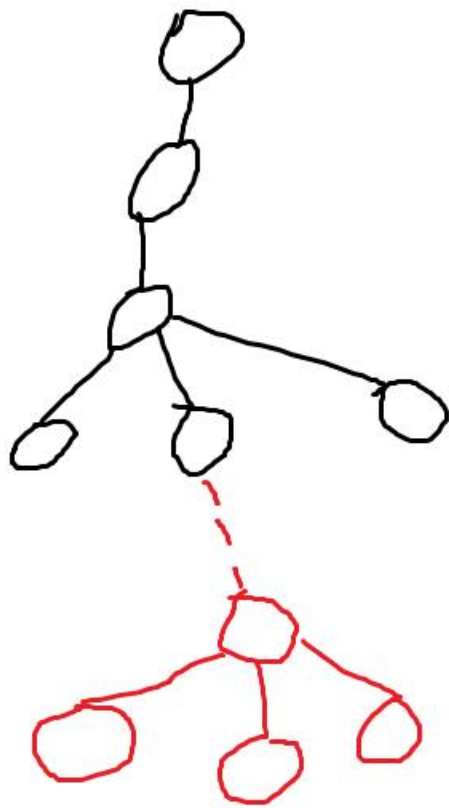
- 通过对每个极大边连通块开一个ETT来维护每个极大边连通块
- 然后每个ETT上面每个节点维护 $O(c)$ 个信息，表示这个节点为根的子树里面有没有节点的儿子为 $1, 2 \dots c$ 这种颜色



Solution2

- 当把这个连通块染色为某个颜色 x 的时候
- while(root的 x 颜色信息为1) 沿着ETT结构递归下去，直到找到一个点其满足至少有一个儿子颜色为 x 颜色
- 然后把这个点的所有颜色为 x 的儿子的ETT合并进这个ETT里面
- 这个通过对每个点开一个 $O(x)$ 的数组来存下每个颜色的儿子节点就可以做到了
- 具体体现在操作上就是合并两棵平衡树，把其中一个平衡树完全合并进另一个平衡树，作为一个区间

Solution2



Solution2

- 这样以 $O(\lg n)$ 代价，我们减少了 $O(1)$ 个联通块
- 每次最多加 $2 = O(1)$ 个联通块
- 所以时间复杂度是均摊 $O((n + m)\lg n)$

Solution2

- 点染色就转化为了边染色，可能会合并连通块，这个是平凡的
- 把x从a染成b，要把x为根的b色连通块和fa(x)为根的b色连通块和边(x,fa(x))合并
- 查询的话，ETT上面维护：最浅，最深的dep，这个联通块的颜色size
- 这样就做完了
- 时间复杂度 $O(n \log n)$ ，空间复杂度 $O(n)$

Solution3

- 发现我们ETT每个节点维护了 $O(c)$ 个信息
 - 这 $O(c)$ 个信息是or起来的
 - 所以可以压位维护
-
- 时间复杂度 $O(n \log n / w)$, 空间复杂度 $O(nc)$

Thanks for listening